Formulario · Programación de Sistemas en Tiempo Real

2026-11-08

T0 · Sistemas de tiempo real

Definiciones clave

- Tarea periódica: (C, T, D) con $D \leq T$ habitual.
- Utilización de una tarea: $U_i = C_i/T_i$.
- Utilización del sistema: $U = \sum_i C_i/T_i$.

T1 · Repaso de C y C++

Plantillas rápidas

- Contar palabras: `++hist[w];`
- ¿Existe la clave?: `if (m.find(k) != m.end())`
- Recorrer mapa: `for (const auto& [k, v] : m)`
- Borrar con condición: `it = v.erase(it);` o `v.erase( remove_if(...), v.end());`
- Ordenar por criterio: `sort(b, e, [](const T& a, const T& b){ return ...; });`

T2 · Clases y objetos

Plantilla de clase para el examen

CPP

```
class C_X {
public:
    C_X(tipo a, tipo b) : a_{a}, b_{b}
     ↪  {}          // constructor
    tipo a() const { return a_; }
     ↪  // getters const
    friend ostream& operator<<(ostream&
     ↪  os, const C_X& x) { return os <<
     ↪  x.a_; }
private:
    tipo a_, b_;
};   // <- el ; final
```

T3 · Herencia y polimorfismo

Plantilla de jerarquía

CPP

```
class Base {
public:
    virtual ~Base() = default;
    virtual double f() const = 0;
     ↪  // pura
    friend ostream& operator<<(ostream&
     ↪  o, const Base& b) { return o <<
     ↪  b.to_string(); }
protected:
    virtual string to_string() const {
     ↪  return "base"; }
};
class Der : public Base {
public:
    double f() const override { return
     ↪  1; }
protected:
    string to_string() const override {
     ↪  return "der " +
     ↪  Base::to_string(); }
};
// uso: vector<Base*> v; for (auto* p :
 ↪  v) cout << *p;
```

T4 · Hilos y exclusión mutua

Plantillas de concurrencia

- Lanzar y esperar: `vector<thread> v; for(...) v.emplace_back(f, args); for (auto& h : v) h.join();`
- Sección crítica: `{ lock_guard<mutex> lk{m}; /* tocar datos protegidos */ }`
- Dos recursos: `scoped_lock lk{m1, m2};`
- Periódica: `auto sig = steady_clock::now() + T; loop { trabajo(); sleep_until(sig); sig += T; }`

T5 · Monitores y variables condición

Plantilla de monitor

CPP

```
class Monitor {
public:
    void operacion() {
        unique_lock<mutex> lk{m_};
        cv_.wait(lk, [this]{ return
         ↪  puedo_; });
        /* actualizar estado */
        cv_.notify_all();
    }
private:
    mutex m_;
    condition_variable cv_;
    /* estado protegido por m_ */
};
```

T6 · Planificación

Planificación: fórmulas clave

- Utilización: $U = \sum C_i/T_i$.
- Cota RM (D=T): $U \leq n(2^{1/n} - 1)$ — $n = 1$: 1; $n = 2$: $0{,}828$; $n = 3$: $0{,}780$.
- EDF (D=T): $U \leq 1$.
- Hiperperiodo: $T_M = \text{mcm}(T_i)$.
- Ciclo secundario: $T_S \mid T_M$, $T_S \geq \max C_i$, $2T_S - \gcd(T_S, T_i) \leq D_i$.
- Tiempo de respuesta: $R_i = C_i + B_i + \sum_{j \in hp(i)} \lceil R_i/T_j \rceil C_j$.
- ICPP: techo$(r) = \max P_k$ de quienes usan r; $B_i = \max c_{k,r}$ con $P_k < P_i$ y techo$(r) \geq P_i$.

T7 · Coma flotante y casos reales

Coma flotante

- `double`: 1 bit de signo + 11 de exponente + 52 de mantisa (≈ 15-16 cifras decimales).
- `float`: 1 + 8 + 23 (≈ 7 cifras decimales).
- Comparar: `fabs(a - b) < tolerancia`.
- Error acumulado ≈ (error por operación) × (número de operaciones).
- Patriot: $e = 0{,}1 - \lfloor 0{,}1 \cdot 2^{23} \rfloor / 2^{23} \approx 9{,}5 \cdot 10^{-8}$ s; en 100 h: 0,34 s.

Prácticas

Resumen de prácticas

- P1: `++hist[p]`, `find` para consultar, `sort` con lambda para ordenar por valor.
- P2: constructor delegado, `operator<<` con `to_string()` virtual, destructor virtual, contenedor recursivo con punteros.
- P3: filas disjuntas ⇒ sin mutex; contador compartido ⇒ mutex; varios recursos ⇒ `scoped_lock`; esperar condición ⇒ `condition_variable` con predicado.